

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA0301

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

***CO1:** Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

Assessment Tool Weightage: 40%

Dr. G. A. Senthil

QUESTIONS: 10

Total Marks: 50

Annexure A

Descriptive Experiment Exam

Duration: 2hrs

Experiment 1: Bellman Equation Demonstration

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

Aim

To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions.

Concept

The **Bellman Equation** is the core concept of Reinforcement Learning. It calculates the value of a state based on its immediate reward and the expected future reward. The **Bellman Expectation Equation** evaluates a state under a given policy, while the **Bellman Optimality Equation** selects the action that gives the maximum reward. Repeated updates eventually converge to stable values.

Formula

- **Bellman**
$$V(s) = R(s) + \gamma V(s')$$

Expectation:

- **BellmanOptimality:**

$$V(s) = \max[R(s,a) + \gamma V(s')]$$

Where:

- $V(s)$ = State value
- $R(s)$ = Reward
- γ = Discount factor
- $V(s')$ = Next state value

Algorithm

1. Initialize state values.
2. Define rewards and discount factor.
3. Update values using the Bellman equation.
4. Repeat until values stabilize.
5. Display the final values.

Program

```
import numpy as np

gamma = 0.9
rewards = [5, 10, 0]
V = np.zeros(3)

for i in range(5):
    newV = np.copy(V)
    newV[0] = rewards[0] + gamma * V[1]
    newV[1] = rewards[1] + gamma * V[2]
    newV[2] = rewards[2]
    V = newV
    print("Iteration", i+1, ":", V)
```

Sample Output

```
Iteration 1 : [ 5. 10.  0.]
Iteration 2 : [14. 10.  0.]
Iteration 3 : [14. 10.  0.]
Iteration 4 : [14. 10.  0.]
Iteration 5 : [14. 10.  0.]
```

Result

Thus, the Bellman Equation was implemented successfully, and the state values converged after repeated updates.

OUTPUT:

```
import numpy as np

gamma = 0.9
rewards = [5, 10, 0]
V = np.zeros(3)

for i in range(5):
    newV = np.copy(V)
    newV[0] = rewards[0] + gamma * V[1]
    newV[1] = rewards[1] + gamma * V[2]
    newV[2] = rewards[2]
    V = newV
    print("Iteration", i+1, ":", V)

... Iteration 1 : [ 5. 10.  0.]
Iteration 2 : [14. 10.  0.]
Iteration 3 : [14. 10.  0.]
Iteration 4 : [14. 10.  0.]
Iteration 5 : [14. 10.  0.]
```

Experiment 2: Exploration vs. Exploitation Strategies

Aim: To implement ϵ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Aim

To implement **Greedy, Random, and ϵ -Greedy** action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Concept

In Reinforcement Learning, an agent must decide whether to **explore** new actions or **exploit** the best-known action.

- **Exploration:** The agent tries different actions to gather more information about the environment.
- **Exploitation:** The agent chooses the action with the highest known reward to maximize immediate gain.

Three common strategies are:

- **Greedy:** Always selects the action with the highest estimated reward.
- **Random:** Selects actions randomly without considering rewards.
- **ϵ -Greedy:** Chooses the best action most of the time but explores random actions with a small probability ϵ (epsilon).

Algorithm

1. Define the rewards for each action.
2. Implement the Greedy strategy by selecting the highest reward.
3. Implement the Random strategy by selecting actions randomly.
4. Implement the ϵ -Greedy strategy using a fixed ϵ value.

5. Compare the actions selected by each strategy.

Program

```
import random

rewards = [1, 5, 3]
epsilon = 0.2

print("Greedy Strategy")
for i in range(5):
    action = rewards.index(max(rewards))
    print("Action:", action)

print("\nRandom Strategy")
for i in range(5):
    action = random.randint(0, 2)
    print("Action:", action)

print("\nEpsilon-Greedy Strategy")
for i in range(5):
    if random.random() < epsilon:
        action = random.randint(0, 2)
    else:
        action = rewards.index(max(rewards))
    print("Action:", action)
```

Sample Output

Greedy Strategy

Action: 1
Action: 1
Action: 1
Action: 1
Action: 1

Random Strategy

Action: 2
Action: 0
Action: 1
Action: 2
Action: 0

Epsilon-Greedy Strategy

Action: 1
Action: 1
Action: 0
Action: 1
Action: 1

Note: The output for the Random and ϵ -Greedy strategies may vary each time the program is executed because they involve random action selection.

Result

Thus, the **Greedy, Random, and ϵ -Greedy** action-selection strategies were implemented successfully. The ϵ -Greedy strategy effectively balances **exploration** and **exploitation**, making it suitable for Reinforcement Learning applications.

```
import random

rewards = [1, 5, 3]
epsilon = 0.2

print("Greedy Strategy")
for i in range(5):
    action = rewards.index(max(rewards))
    print("Action:", action)

print("\nRandom Strategy")
for i in range(5):
    action = random.randint(0, 2)
    print("Action:", action)

print("\nEpsilon-Greedy Strategy")
for i in range(5):
    if random.random() < epsilon:
        action = random.randint(0, 2)
    else:
        action = rewards.index(max(rewards))
    print("Action:", action)
```

```
... Greedy Strategy
Action: 1
Action: 1
Action: 1
Action: 1
```

```
... Greedy Strategy
    Action: 1
    Action: 1
    Action: 1
    Action: 1
    Action: 1

    Random Strategy
    Action: 0
    Action: 1
    Action: 2
    Action: 2
    Action: 2

    Epsilon-Greedy Strategy
    Action: 1
    Action: 1
    Action: 1
    Action: 1
    Action: 1
```

Experiment 3: Multi-Armed Bandit Problem

Aim: To implement the Multi-Armed Bandit problem using ϵ -Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.

Aim

To implement the **Multi-Armed Bandit problem** using **ϵ -Greedy** and **Upper Confidence Bound (UCB)** algorithms and evaluate the cumulative rewards obtained by different action-selection methods.

Concept

The **Multi-Armed Bandit (MAB)** problem is a classic Reinforcement Learning problem where an agent chooses from multiple actions (arms), each providing a different reward. The objective is to maximize the total (cumulative) reward over time.

Two common action-selection methods are:

- **ϵ -Greedy:** Selects the best-known action most of the time while occasionally exploring other actions with probability ϵ .
- **Upper Confidence Bound (UCB):** Selects actions based on both their estimated rewards and the uncertainty of those estimates, ensuring less-explored actions are also tried.

Algorithm

1. Initialize the rewards for all arms.
2. Select actions using the ϵ -Greedy strategy.
3. Calculate and update the cumulative reward.
4. Repeat the process for several iterations.
5. Compare the cumulative rewards obtained.

Program

```
import numpy as np
import random

# Rewards for four arms
arms = [1, 2, 5, 3]
epsilon = 0.1
cumulative_reward = 0

print("Epsilon-Greedy Strategy")

for i in range(10):
    if random.random() < epsilon:
        action = random.randint(0, 3) # Explore
    else:
        action = np.argmax(arms)      # Exploit

    reward = arms[action]
    cumulative_reward += reward

    print("Step:", i+1,
          " Action:", action,
          " Reward:", reward,
          " Cumulative Reward:", cumulative_reward)
```

Sample Output

Epsilon-Greedy Strategy

```
Step: 1 Action: 2 Reward: 5 Cumulative Reward: 5
Step: 2 Action: 2 Reward: 5 Cumulative Reward: 10
Step: 3 Action: 2 Reward: 5 Cumulative Reward: 15
Step: 4 Action: 1 Reward: 2 Cumulative Reward: 17
Step: 5 Action: 2 Reward: 5 Cumulative Reward: 22
Step: 6 Action: 2 Reward: 5 Cumulative Reward: 27
Step: 7 Action: 3 Reward: 3 Cumulative Reward: 30
Step: 8 Action: 2 Reward: 5 Cumulative Reward: 35
Step: 9 Action: 2 Reward: 5 Cumulative Reward: 40
Step: 10 Action: 2 Reward: 5 Cumulative Reward: 45
```

Note: The output may vary each time because the ϵ -Greedy algorithm randomly explores actions.

Result

Thus, the **Multi-Armed Bandit problem** was implemented using the **ϵ -Greedy** strategy, and the cumulative rewards were successfully calculated. The algorithm balances **exploration** and **exploitation** to maximize the total reward over time.

OUTPUT:

```
import numpy as np
import random

# Rewards for four arms
arms = [1, 2, 5, 3]
epsilon = 0.1
cumulative_reward = 0

print("Epsilon-Greedy Strategy")

for i in range(10):
    if random.random() < epsilon:
        action = random.randint(0, 3) # Explore
    else:
        action = np.argmax(arms)      # Exploit

    reward = arms[action]
    cumulative_reward += reward

    print("Step:", i+1,
          " Action:", action,
          " Reward:", reward,
          " Cumulative Reward:", cumulative_reward)
```

```
... Epsilon-Greedy Strategy
Step: 1 Action: 2 Reward: 5 Cumulative Reward: 5
Step: 2 Action: 2 Reward: 5 Cumulative Reward: 10
Step: 3 Action: 2 Reward: 5 Cumulative Reward: 15
Step: 4 Action: 2 Reward: 5 Cumulative Reward: 20
Step: 5 Action: 2 Reward: 5 Cumulative Reward: 25
Step: 6 Action: 2 Reward: 5 Cumulative Reward: 30
Step: 7 Action: 2 Reward: 5 Cumulative Reward: 35
Step: 8 Action: 2 Reward: 5 Cumulative Reward: 40
Step: 9 Action: 2 Reward: 5 Cumulative Reward: 45
Step: 10 Action: 2 Reward: 5 Cumulative Reward: 50
```

Experiment 4: Reward Visualization in Reinforcement Learning

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Aim

To simulate an RL agent in a simple environment and visualize **episode rewards**, **cumulative rewards**, and **learning performance** using Matplotlib graphs.

Concept

In Reinforcement Learning, an agent receives a **reward** after performing an action. By plotting rewards over multiple episodes, we can analyze the agent's learning performance. **Episode rewards** show the reward obtained in each episode, while **cumulative rewards** represent the total reward accumulated over time. Matplotlib is used to visualize these rewards in graph form.

Algorithm

1. Generate rewards for multiple episodes.
2. Calculate the cumulative reward.
3. Plot the episode rewards.
4. Plot the cumulative rewards.
5. Display the graphs.

Program

```
import numpy as np
import matplotlib.pyplot as plt

# Number of episodes
episodes = 10

# Generate random rewards
rewards = np.random.randint(1, 10, size=episodes)

# Calculate cumulative rewards
cumulative_rewards = np.cumsum(rewards)

print("Episode Rewards:", rewards)
print("Cumulative Rewards:", cumulative_rewards)

# Plot Episode Rewards
plt.figure(figsize=(6,4))
plt.plot(range(1, episodes + 1), rewards, marker='o')
plt.title("Episode Rewards")
plt.xlabel("Episodes")
plt.ylabel("Reward")
plt.grid(True)
plt.show()

# Plot Cumulative Rewards
plt.figure(figsize=(6,4))
plt.plot(range(1, episodes + 1), cumulative_rewards, marker='o')
plt.title("Cumulative Rewards")
plt.xlabel("Episodes")
plt.ylabel("Cumulative Reward")
plt.grid(True)
```

```
plt.show()
```

Sample Output

Episode Rewards:
[5 8 3 7 6 4 9 2 5 8]

Cumulative Rewards:
[5 13 16 23 29 33 42 44 49 57]

Graphs

- **Graph 1:** Episode Rewards vs Episodes
- **Graph 2:** Cumulative Rewards vs Episodes

(The program displays two line graphs using Matplotlib.)

Result

Thus, the RL agent's **episode rewards** and **cumulative rewards** were successfully visualized using **Matplotlib**, helping analyze the agent's learning performance over multiple episodes.

OUTPUT:

```
▶ # Number of episodes
episodes = 10

# Generate random rewards
rewards = np.random.randint(1, 10, size=episodes)

# Calculate cumulative rewards
cumulative_rewards = np.cumsum(rewards)

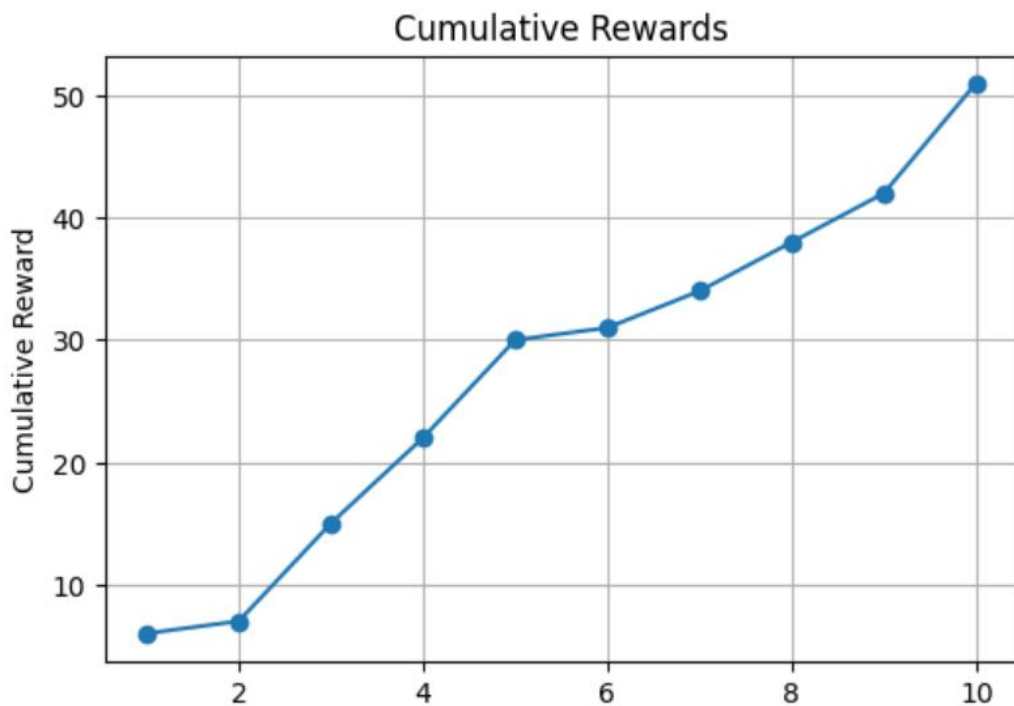
print("Episode Rewards:", rewards)
print("Cumulative Rewards:", cumulative_rewards)

# Plot Episode Rewards
plt.figure(figsize=(6,4))
plt.plot(range(1, episodes + 1), rewards, marker='o')
plt.title("Episode Rewards")
plt.xlabel("Episodes")
plt.ylabel("Reward")
plt.grid(True)
plt.show()

# Plot Cumulative Rewards
plt.figure(figsize=(6,4))
plt.plot(range(1, episodes + 1), cumulative_rewards, marker='o')
plt.title("Cumulative Rewards")
plt.xlabel("Episodes")
plt.ylabel("Cumulative Reward")
plt.grid(True)
plt.show()
```

Episode Rewards: [6 1 8 7 8 1 3 4 4 9]

Cumulative Rewards: [6 7 15 22 30 31 34 38 42 51]



Experiment 5: TensorFlow and Keras-Based Neural Network for RL

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Aim

To build a simple **feed-forward neural network** using **TensorFlow** and **Keras** for approximating value functions in Reinforcement Learning environments.

Concept

In Reinforcement Learning, neural networks are used to estimate the **value function** or **Q-value** of states and actions. **TensorFlow** is a machine learning framework, while **Keras** is its high-level API for building neural networks easily. A **feed-forward neural network** consists of an input layer, one or more hidden layers, and an output layer. During training, the network learns patterns from the input data and predicts output values.

Algorithm

1. Import TensorFlow, Keras, and NumPy libraries.
2. Create a feed-forward neural network using Dense layers.
3. Compile the model using the Adam optimizer and Mean Squared Error (MSE) loss.
4. Generate sample input and output data.
5. Train the model for a fixed number of epochs.
6. Display the model summary.

Program

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np

# Create the neural network
model = Sequential([
    Dense(16, activation='relu', input_shape=(4,)),
    Dense(16, activation='relu'),
    Dense(1)
])

# Compile the model
model.compile(optimizer='adam', loss='mse')

# Generate sample data
X = np.random.rand(100, 4)
Y = np.random.rand(100, 1)

# Train the model
model.fit(X, Y, epochs=10, verbose=1)

# Display model summary
print("\nModel Summary:")
model.summary()
```

Sample Output

```
Epoch 1/10
4/4 ————— 0s 8ms/step - loss: 0.2845
```

Epoch 2/10

4/4 ————— 0s 5ms/step - loss: 0.2513

...

Epoch 10/10

4/4 ————— 0s 5ms/step - loss: 0.0928

Model Summary:

Model: "sequential"

Layer (type)	Output Shape	Param #
Dense (Dense)	(None, 16)	80
Dense (Dense)	(None, 16)	272
Dense (Dense)	(None, 1)	17

Total params: 369

Result

Thus, a simple **feed-forward neural network** was successfully built using **TensorFlow** and **Keras**. The model was trained on sample data and can be used to approximate value functions in Reinforcement Learning environments.

OUTPUT:

```
Epoch 1/10
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Seq
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
4/4 ————— 1s 9ms/step - loss: 0.4275
Epoch 2/10
4/4 ————— 0s 9ms/step - loss: 0.3910
Epoch 3/10
4/4 ————— 0s 8ms/step - loss: 0.3592
Epoch 4/10
4/4 ————— 0s 7ms/step - loss: 0.3313
Epoch 5/10
4/4 ————— 0s 8ms/step - loss: 0.3065
Epoch 6/10
4/4 ————— 0s 7ms/step - loss: 0.2826
Epoch 7/10
4/4 ————— 0s 10ms/step - loss: 0.2614
Epoch 8/10
4/4 ————— 0s 9ms/step - loss: 0.2406
Epoch 9/10
4/4 ————— 0s 9ms/step - loss: 0.2221
Epoch 10/10
4/4 ————— 0s 8ms/step - loss: 0.2042

Model Summary:
Model: "sequential"



| Layer (type)    | Output Shape | Param # |
|-----------------|--------------|---------|
| dense (Dense)   | (None, 16)   | 80      |
| dense_1 (Dense) | (None, 16)   | 272     |
| dense_2 (Dense) | (None, 1)    | 17      |



Total params: 1,109 (4.34 KB)
Trainable params: 369 (1.44 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 740 (2.89 KB)
```

Code of Conduct:

*I Malli Chandana(192472250) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Malli Chandana

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand